

use.ai

Start new

Files & documents

Projects

Images

Apps

My organization

My chats

Vendor Management System

Share Feedback

Priyanshi Sharma

priyanshi.25.12.1980@gmail...

GPT 5.4

Copy chat

Download chat

Share

Go Pro

I have a requirement to make an HRMS and vendor management kind of system. The situation is like this: one PSU company hires vendor employees through NICSi, and they want a way to manage purchase orders, Work orders, Advance payments, vendor employee details, and attendance.

I happens like this: Suppose there are multiple projects in the company and there is a requirement for more manpower, so the project manager can raise a purchase order request through SAP, and the purchase department will handle the purchase order process; we only need to record and track the Purchase order number(PO No.) and other basic PO details, the NICSi will share the CVs of potential employees for hiring and the PSU will select the people after interview, When the selection is done NICSi will tell the price for hiring the people, We need to pay the advance salary for hiring the people to NICSi (ex:- we want to hire 2 people for 6 months with a salary of 1 lakh each, so we'll have to pay 12 lakh + TDS), Here we need to find a way to track the advance details, TDS details, advance details will be tracked through bank reference number or the check number. Once the Advance has been paid, NICSi will release a work order(WO) for the advance paid by the PSU. It'll have the resource details, advance paid details, and total. But NICSi only gives one Project ID for the PSU subunit. But the PSU wants to have separate project IDs for different projects. After the work order is released, the vendor employees can come to work in the PSU; now we need to manage their attendance and record their details like basics, CV, PF, insurance, and all in master data. Then add the vendor employees to a specific Project ID. At the end of each month, the PSU will give one attendance certificate to NICSi, which will in turn give the attendance certificate to the vendor employee's agency; then they'll raise one invoice to NICSi, and NICSi will pay the salary to the vendors, and then NICSi will raise one invoice to the PSU; the advance with NICSi will get reduced. Now the PSU needs to monitor and record the advance with NICSi based on the specific project and also see the overall advance with NICSi for all projects combined. Here, only two logins will be there: the admin and the project manager. One project manager can have multiple projects under him, and one Project (project ID) can have multiple POs and WOs, but one PO will only have one Work order (they have a one-to-one relationship), and the advance has to be attached to the PO and WO. And for each project, the vendor details should also be attached to the work order and the purchase order. Dashboards are required to monitor the advance and the attendance. Suppose after 6 months, the WO has expired and only 1 lakh is left of the 12 lakh advance (amount left because of the vendor employee taking leave or other reasons), and there is still more manpower required, so again a new PO will be requested and again a new Work order will be released, and a new advance will be paid. And the cycle will keep repeating, hence there being multiple PO no. and WO no. for a project.

I need help designing the whole data flow, workflow, and the architecture of the project.

Design it around **three things you must always be able to answer:**

1. How much advance is lying with NICSi?

2. Which vendor employees are deployed on which project/PO/WO?

3. What attendance-certified consumption has reduced that advance?

If the system can answer those reliably at **project level**, **PO/WO level**, and **overall NICSi level**, the rest is mostly workflow and reporting.

1) Core business understanding

From your description, the process is:

1. A **Project Manager** raises manpower requirement.

2. A **PO** is created through SAP, but your system only tracks the PO reference and business details.

3. **NICSi** shares candidate CVs.

4. PSU selects candidates.

5. NICSi shares commercial value for deployment.

6. PSU pays **advance** to NICSi against the PO/hiring period.

7. NICSi issues **WO** against that advance.

8. Vendor employees join and are mapped to PSU projects.

9. Attendance is captured monthly.

10. PSU issues attendance certificate.

11. Vendor agency invoices NICSi, NICSi pays vendor staff, NICSi raises invoice to PSU.

12. Monthly consumption reduces the **advance balance**.

13. On expiry, remaining advance may still exist; then a fresh PO/WO/advance cycle starts.

So this is not just HRMS. It is a **Vendor Resource Management + Advance Ledger + Attendance Certification + PO/WO Tracking** system.

2) Recommended module structure

Build the system as these modules:

A. Master Data

- Projects
- Users
- Vendors / agencies
- NICS master
- Employee skill/designation/rate categories
- Cost centers / departments / business units
- Document types

B. PO / WO Management

- Purchase Order tracking
- Work Order tracking
- PO-WO linkage
- Project mapping
- Resource commitment details

C. Advance Management

- Advance request details
- Payment details
- TDS details
- Bank / cheque reference
- Advance ledger
- Adjustment / utilization against attendance-certified monthly billing

D. Vendor Employee Management

- Candidate CV pipeline
- Selected candidate records
- Joining records
- Employee master
- Compliance documents: PF, ESI/insurance, Aadhaar/PAN if required by policy, bank details if needed, CV, offer/reference docs
- Deployment history across projects and WO periods

E. Attendance & Certification

- Daily/monthly attendance capture
- Approval by project manager
- Attendance certificate generation
- Locking and versioning

F. Dashboard & Reporting

- Advance balance by project
- Advance balance by PO/WO
- Overall NICS balance
- Deployed headcount
- Attendance summary
- Expiring WO/resources
- Unadjusted advances / low balances

G. Audit & Document Repository

- Uploaded files
- Change logs
- Approval history
- Downloadable certificates and registers

3) High-level business entities

These are your main entities and relationships.

Main entities

- User
- Role

- Project
- Project Manager Assignment
- Vendor
- NICS Agreement / Account
- Purchase Order
- Work Order
- Advance Payment
- Advance Ledger Entry
- Candidate
- Vendor Employee
- Employee Deployment
- Attendance Period
- Attendance Record
- Attendance Certificate
- Invoice / Consumption Entry
- Document

4) Correct data model relationships

You mentioned:

- One **Project Manager** can manage multiple **Projects**
- One **Project** can have multiple **POs**
- One **Project** can have multiple **WOs**
- One **PO** has one **WO** only
- Advance must be attached to **PO** and **WO**
- Vendor employees should be attached to **Project**, **PO**, and **WO**

That leads to this practical model:

Relationship summary

- User 1..N Project
- Project 1..N PO
- PO 1..1 WO
- Project 1..N WO
- PO 1..N Advance Payment
Even if business usually pays one advance per PO, keep it 1..N in the database because split payments, corrections, or partial releases happen in real life.
- WO 1..N Employee Deployment
- Vendor Employee 1..N Employee Deployment
- WO 1..N Attendance Period
- Attendance Period 1..N Attendance Records
- WO 1..N Invoice/Consumption Entries
- Advance Ledger links to PO, WO, Project, and monthly consumption

Important design advice

Do **not** hardcode advance as a single field inside PO or WO.

Instead, maintain a **ledger-based model**:

- Credit entries = advance payments made to NICS
- Debit entries = monthly utilization/consumption against attendance-certified billing
- Adjustments = refunds, corrections, carry forward, TDS reconciliation if needed

That is the cleanest way to calculate:

- balance by **project**
- balance by **PO**
- balance by **WO**
- balance by **NICS overall**

5) Recommended workflow

5.1 Requirement to PO tracking

1. Project Manager creates **Manpower Requisition**
 - Project
 - required roles
 - count
 - duration
 - estimated rate

- justification

2. Admin records/updates **PO details from SAP**

- PO No
- PO date
- SAP reference
- project
- manpower period
- total PO value
- remarks

Even if SAP remains source of truth for PO approval, your system should keep a mirrored PO tracking record.

5.2 Candidate selection workflow

1. NICS I shares CVs
2. Admin uploads CVs under candidate pool
3. Project Manager reviews
4. Interview status updated
5. Candidate selected / rejected / hold
6. On selection, candidate becomes **Vendor Employee (pending joining)**

Useful statuses:

- CV Received
- Under Review
- Interview Scheduled
- Selected
- Rejected
- Offered by NICS I
- Joined
- Not Joined
- Replaced

5.3 Commercial and advance workflow

1. For selected resources, NICS I shares rates
2. Admin enters commercial details:
 - role
 - salary/rate
 - count
 - months
 - total value
 - taxes / TDS
3. Advance advice generated
4. PSU pays advance
5. Payment details recorded:
 - payment date
 - gross advance
 - TDS
 - net paid
 - cheque / bank UTR / transaction ref
6. Ledger creates **credit entry**
7. WO is expected from NICS I
8. Admin records WO details against same PO

Example

For 2 employees x 1,00,000 × 6 months = 12,00,000

- gross advance = 12,00,000
- TDS = maybe as applicable
- net paid = actual transfer
- ledger should store all components clearly

Important:

- Keep **gross contractual advance** and **actual payment movement** separate.
- TDS should not vanish inside net amount. It must remain visible for reconciliation.

5.4 Work order workflow

Once NICS issues WO:

- WO No
- WO date
- linked PO
- linked project
- NICS master Project ID
- PSU internal Project ID
- validity start/end
- covered resources
- covered amount
- reference to advance

Critical requirement: dual project coding

Because NICS provides only one Project ID for the PSU subunit, but PSU needs multiple internal project IDs, store both:

- NICS Project Code
- PSU Internal Project ID

And allow one NICS project code to map to many internal projects if required.

5.5 Vendor employee onboarding and deployment

Once WO is active and people join:

Capture employee master:

- employee code (internal)
- name
- agency/vendor
- NICS reference
- designation
- skill category
- DOJ
- mobile
- email
- Aadhaar/PAN if policy permits
- PF number
- ESI/insurance details
- bank if needed
- CV document
- KYC/compliance documents
- status active/inactive/exited

Then create **deployment records**, not just one-time mapping:

- employee
- project
- PO
- WO
- start date
- end date
- role
- billing rate
- reporting manager
- work location

Why deployment table matters: A vendor employee may work across:

- different WOs over time
- the same project under renewed contracts
- replacement scenarios

Without deployment history, reporting becomes messy.

5.6 Attendance workflow

Recommended monthly structure:

1. Attendance month created for active deployments

2. Daily attendance marked or imported
3. Monthly summary generated:
 - present
 - absent
 - leave
 - holidays
 - billable days
4. Project Manager verifies
5. Admin locks attendance
6. Attendance certificate generated
7. Certificate shared with NICSi
8. Certified attendance becomes basis for monthly utilization against advance

Key point

Attendance should reduce advance only **after certification**, not merely after raw attendance entry.

5.7 Advance consumption workflow

At month end:

1. Attendance certificate approved
2. Billing/consumption amount calculated per employee or per WO
3. System posts **ledger debit**
4. Advance balance decreases

This gives a clean formula:

$$\text{Advance Balance} = \sum \text{Advance Credits} - \sum \text{Consumption Debits} \pm \sum \text{Adjustments}$$

Example

Initial advance credit:

- 12,00,000

Month 1 salary utilization:

- 2,00,000 debit

Month 2:

- 1,80,000 debit because leave deduction

And so on.

After 6 months, if only 11,00,000 has been consumed:

$$12,00,000 - 11,00,000 = 1,00,000$$

That remaining 1,00,000 should remain visible as:

- balance under that **WO**
- balance under that **PO**
- balance under that **project**
- balance under NICSi overall

6) Recommended state/status model

Statuses make the workflow trackable.

PO status

- Draft
- Requested
- PO Received
- Advance Pending
- Advance Partially Paid
- Advance Paid
- WO Pending
- WO Issued
- Active
- Closed
- Cancelled

WO status

- Pending

- Issued
- Active
- Expiring Soon
- Expired
- Closed

Employee status

- Candidate
- Selected
- Pending Joining
- Active
- On Hold
- Replaced
- Exited

Attendance status

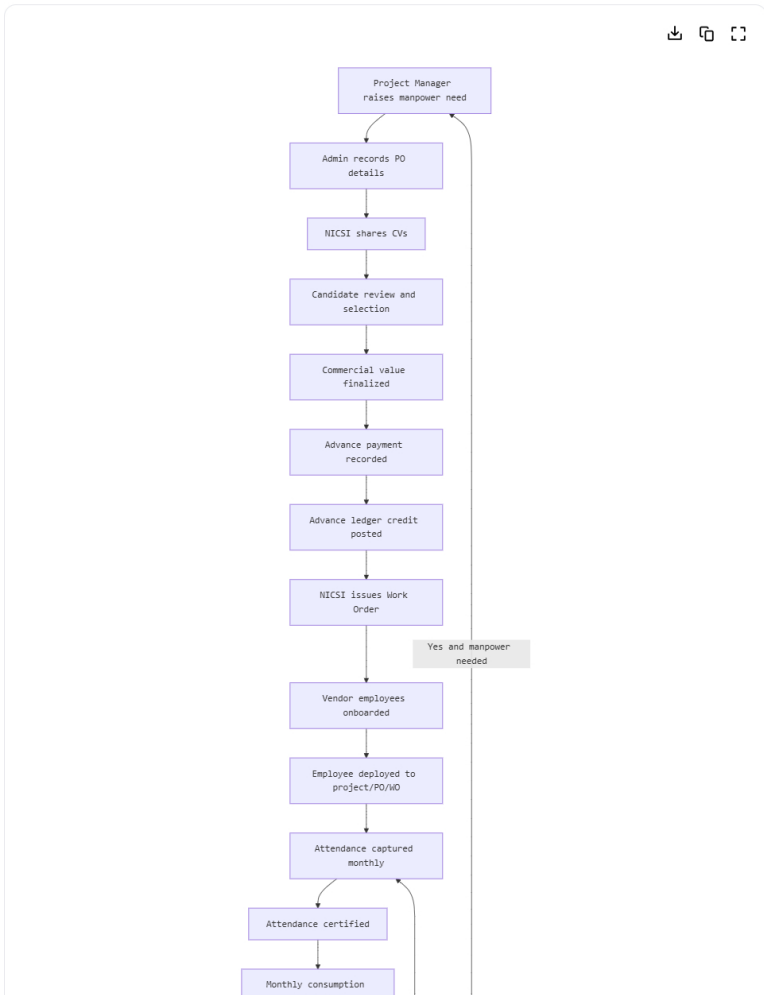
- Draft
- Submitted
- Approved by PM
- Locked
- Certified
- Billed/Adjusted

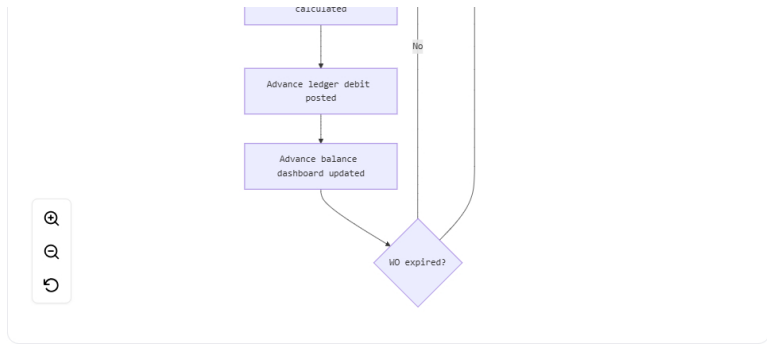
Advance entry status

- Initiated
- Paid
- Verified
- Partially Adjusted
- Fully Adjusted
- Refunded / Carried Forward / Closed

7) Data flow design

Here is the cleanest high-level data flow.





8) Suggested database design

Below is a practical relational design.

8.1 Users and access

users

- id
- name
- email
- mobile
- password_hash / SSO id
- role_id
- status
- created_at

roles

- id
 - name
- Values: Admin, Project Manager

project_manager_projects

- id
- user_id
- project_id

8.2 Project master

projects

- id
- internal_project_id
- project_name
- department
- cost_center
- nicsi_project_code
- description
- start_date
- end_date
- status

If one NICSi code can apply across many PSU projects, store it here or in a separate mapping table.

nicsi_project_mappings

- id
- nicsi_project_code
- project_id
- active_from
- active_to

8.3 Vendor and agency master

vendors

- id
- vendor_name
- vendor_type

Values: NICSI, Agency

- address
- contact_person
- email
- mobile
- gstin
- pan
- status

You may keep NICSI as one special vendor record and other staffing agencies separately.

8.4 PO and WO

purchase_orders

- id
- po_no
- po_date
- sap_reference_no
- project_id
- pm_user_id
- vendor_id
likely NICSI
- manpower_start_date
- manpower_end_date
- po_value
- currency
- description
- status
- remarks

work_orders

- id
- wo_no
- wo_date
- po_id
- project_id
- nicsi_project_code
- wo_start_date
- wo_end_date
- wo_value
- status
- remarks

Put a unique constraint on po_id in work_orders if PO truly has only one WO.

8.5 Candidate and employee

candidates

- id
- vendor_id
- source_ref_no
- name
- email
- mobile
- designation
- skill_category
- experience_years
- cv_document_id
- status
- remarks

candidate_interviews

- id
- candidate_id
- project_id
- interview_date
- interviewer_name

- result
 - notes
- vendor_employees**
- id
 - candidate_id nullable
 - employee_code
 - vendor_id
 - name
 - email
 - mobile
 - designation
 - skill_category
 - date_of_joining
 - date_of_exit
 - pf_no
 - esi_no
 - insurance_policy_no
 - pan_no
 - aadhaar_no
 - status
-

8.6 Deployment

- employee_deployments**
- id
 - vendor_employee_id
 - project_id
 - po_id
 - wo_id
 - billing_vendor_id
 - role_name
 - monthly_rate
 - deployment_start_date
 - deployment_end_date
 - reporting_manager_user_id
 - status

This is one of the most important tables.

8.7 Advance and ledger

- advance_payments**
- id
 - project_id
 - po_id
 - wo_id nullable until WO created
 - vendor_id
 - payment_date
 - payment_mode
 - bank_name
 - bank_reference_no
 - cheque_no
 - gross_advance_amount
 - tds_amount
 - other_deductions
 - net_paid_amount
 - narration
 - status
 - attachment_document_id

- advance_ledger_entries**
- id
 - project_id
 - po_id
 - wo_id

- vendor_id
- entry_date
- entry_type
Values: CREDIT_ADVANCE, DEBIT_UTILIZATION, CREDIT_REFUND, DEBIT_ADJUSTMENT, CREDIT_CARRY_FORWARD
- reference_type
Values: ADVANCE_PAYMENT, ATTENDANCE_CERTIFICATE, MANUAL_ADJUSTMENT, OPENING_BALANCE
- reference_id
- amount
- remarks
- created_by

This is the accounting heart of the application.

8.8 Attendance

attendance_periods

- id
- project_id
- po_id
- wo_id
- month
- year
- period_start
- period_end
- status
- certified_on
- certified_by

attendance_records

- id
- attendance_period_id
- vendor_employee_id
- present_days
- absent_days
- leave_days
- holiday_days
- billable_days
- overtime_days nullable
- remarks

If daily attendance is needed:

daily_attendance

- id
- vendor_employee_id
- project_id
- attendance_date
- status
- in_time nullable
- out_time nullable
- remarks

Then monthly records can be generated from daily data.

8.9 Attendance certification and utilization

attendance_certificates

- id
- attendance_period_id
- certificate_no
- certificate_date
- total_employees
- total_billable_amount
- status
- document_id

utilization_entries

utilization_entries

- id
- attendance_certificate_id
- project_id
- po_id
- wo_id
- vendor_employee_id nullable
- amount
- remarks

Each utilization entry can trigger corresponding ledger debits.

8.10 Documents

documents

- id
- entity_type
- entity_id
- document_type
- file_name
- file_path
- mime_type
- uploaded_by
- uploaded_at
- version_no

9) Ledger logic for advance monitoring

This part is the most important because the PSU explicitly wants:

- project-wise advance
- overall NICS I advance
- PO-wise and WO-wise tracking

Example ledger

Date	Project	PO	WO	Entry Type	Amount	Effect
01-Apr	P1	PO-101	WO-501	CREDIT_ADVANCE	12,00,000	+
30-Apr	P1	PO-101	WO-501	DEBIT_UTILIZATION	2,00,000	-
31-May	P1	PO-101	WO-501	DEBIT_UTILIZATION	1,80,000	-
30-Jun	P1	PO-101	WO-501	DEBIT_UTILIZATION	2,00,000	-

Balance can be grouped by any dimension.

Key reports from ledger

- Advance balance by project
- Advance balance by WO
- Advance aging
- Advances nearing exhaustion
- Expired WO with unspent balance
- New cycle comparison: old WO leftover vs new WO fresh advance

10) Dashboards to build

10.1 Admin dashboard

- Total active projects
- Total active POs
- Total active WOs
- Total vendor employees deployed
- Total advance paid to NICS I
- Total advance utilized
- Current advance balance
- Expired WOs with residual balance
- WOs expiring in next 30 days
- Attendance certification pending count

10.2 Project manager dashboard

- My projects
- Active deployed resources by project
- Current month attendance pending approval
- Project-wise advance:
 - total credited
 - utilized
 - available
- WO expiry alerts
- Resource shortages / vacancies
- Employees with missing documents

Best dashboard cards

For each project show:

- sanctioned headcount
- deployed headcount
- current WO period
- current advance balance
- current month certified billable amount
- last attendance certificate date

11) Key business rules to enforce

These rules will save you from bad data.

1. **One PO can have only one WO**
Enforce using unique constraint on WO.po_id.
2. **One project can have many PO/WO cycles**
Allow multiple PO records per project.
3. **Advance cannot be utilized beyond available balance**
Unless explicit override is allowed to show negative balance.
4. **Attendance certificate must be locked before utilization posting**
5. **Employee deployment dates must fall within WO validity**, unless exception is allowed.
6. **A vendor employee cannot be deployed to overlapping active assignments** on the same dates unless business permits split deployment.
7. **WO cannot become active until advance is paid/verified**, if that is your business rule.
8. **Residual balance on expired WO** must remain traceable.
Decide whether it:
 - stays under old WO only
 - is carried forward to project pool
 - is manually adjusted to new WO
9. **TDS and gross advance must both be reportable**
10. **All critical uploads should be versioned**
 - CV
 - WO copy
 - attendance certificate
 - payment proof

12) Carry-forward design decision

You need to decide one policy for leftover advance after WO expiry.

Option A: Keep residual under old WO

- Best for audit clarity
- New WO gets fresh advance
- Reports show old WO leftover separately

Option B: Carry residual to next WO

- Better for operational balance tracking
- Needs explicit ledger transfer entry:
 - debit old WO
 - credit new WO

Recommendation

Use **ledger transfer entries** so both audit and business visibility remain intact.

Example:

- Old WO residual: 1,00,000

- New WO fresh advance: 12,00,000
- Carry forward 1,00,000
- New WO visible balance: 13,00,000
- Old WO closes at zero after transfer

That requires entry types like:

- DEBIT_TRANSFER_OUT
- CREDIT_TRANSFER_IN

13) Proposed application architecture

For this kind of system, a standard enterprise web app is enough.

13.1 Suggested architecture

- **Frontend:** React / Angular
- **Backend API:** Java Spring Boot or .NET or Node.js NestJS
- **Database:** PostgreSQL or MS SQL Server
- **File storage:** S3 compatible storage / Azure Blob / on-prem file server
- **Auth:** internal login or SSO/LDAP
- **Reporting:** built-in dashboard + export to Excel/PDF

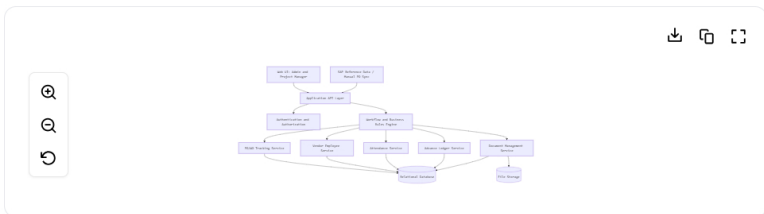
My recommendation

If this is a PSU-style internal enterprise app:

- **Backend:** Spring Boot
- **Frontend:** Angular or React
- **DB:** PostgreSQL
- **Reports:** JasperReports / server-side PDF + Excel export

That stack is stable and audit-friendly.

14) Layered system architecture



You do not necessarily need microservices. A **modular monolith** is better at first.

15) Why modular monolith is better than microservices here

Use a **modular monolith** unless this system is expected to become very large.

Why:

- only 2 role types
- straightforward business process
- strong transactional requirements
- reports need cross-module joins
- audit and consistency are more important than extreme scale

Modules in one codebase:

- auth
- masters
- PO/WO
- employees
- attendance
- ledger
- dashboard
- documents

Later, if needed, you can split out:

- document service
- reporting service

16) Suggested screen list

Admin screens

- Dashboard
- Project master
- User-project mapping
- Vendor master
- PO register
- WO register
- Advance payment entry
- Advance ledger view
- Candidate/CV management
- Vendor employee master
- Deployment mapping
- Attendance month setup
- Attendance certificate generation
- Utilization posting
- Reports
- Document repository

Project manager screens

- My dashboard
- My projects
- Manpower requisition
- Candidate review/interview status
- Employee deployment list
- Attendance approval
- Project advance balance
- WO/PO history
- Project documents

17) Suggested approval flow with only two roles

Since only **Admin** and **Project Manager** exist, keep workflow simple.

Project Manager

- raise requirement
- review CV
- mark interview result
- approve attendance
- view project balances and deployed staff

Admin

- maintain masters
- enter PO/WO
- record advance payment
- onboard employees
- upload compliance docs
- generate attendance certificate
- post monthly utilization
- maintain reports and dashboards

18) Integration approach with SAP

Since you said PO is created in SAP and you only need to track it:

Option 1: Manual entry

Admin enters:

- PO No.
- date
- project
- amount
- SAP reference

Good for phase 1.

Option 2: SAP import

Periodic import via:

- CSV upload

- API
- scheduled file exchange

Recommendation: start with **manual entry + Excel import**, then automate later.

19) Audit and compliance features you should not skip

For a PSU/internal audit environment, these are important:

- created by / created at / updated by / updated at
- status transition history
- document versioning
- no hard delete on financial records
- soft delete / archive only
- approval timestamps
- downloadable audit trail for PO, WO, advance, attendance certificate
- maker-checker style optional approval, even if only two roles exist

20) Recommended reporting pack

Build these reports first:

Financial

- Advance register
- TDS register
- Project-wise advance balance
- WO-wise advance balance
- PO-wise advance paid vs utilized
- NICS consolidated balance
- carry-forward / transfer register

HR / resource

- deployed employees by project
- employee compliance status
- employees by vendor agency
- joining and exit register
- WO-wise employee deployment

Attendance

- monthly attendance summary
- attendance certificate register
- employee-wise billable days
- project-wise attendance trend

Control reports

- expired WO with active employees
- active employees without valid WO
- attendance pending approval
- employees with missing PF/insurance docs
- negative or over-utilized advance exceptions

21) Recommended implementation phases

Phase 1

- login and roles
- project master
- PO tracking
- WO tracking
- vendor employee master
- deployment mapping
- advance payment entry
- advance ledger
- basic dashboard

Phase 2

- candidate CV management
- attendance capture
- attendance approval

- attendance certificate
- utilization posting
- project-wise reports

Phase 3

- SAP import
- document versioning improvements
- alerts and notifications
- carry-forward automation
- stronger audit/reporting exports

22) A practical end-to-end example

Project A

- Internal Project ID: P-A001
- NICS I Project Code: NICS I-UNIT-01

Cycle 1

- PO: PO-1001
- WO: WO-2001
- 2 resources
- 6 months
- monthly salary per resource: 1,00,000
- advance: 12,00,000
- TDS recorded separately

Ledger:

- credit 12,00,000 on advance payment

Monthly certified utilization:

- Apr: 2,00,000
- May: 1,80,000
- Jun: 2,00,000
- Jul: 2,00,000
- Aug: 1,90,000
- Sep: 1,30,000

Total utilized = 11,00,000

Balance = 1,00,000

Cycle 2

- PO: PO-1002
- WO: WO-2002
- fresh advance: 8,00,000

Now reports should show:

Project balance

- Old WO residual: 1,00,000
- New WO fresh: 8,00,000
- Total project advance available: 9,00,000

Or if transferred:

- Old WO closes at 0
- New WO opening available: 9,00,000

23) The biggest design mistake to avoid

Do **not** model this as:

- one PO table
- one employee table
- one attendance table
- one balance field

That will break as soon as:

- WO renews
- leftover advance exists
- one employee shifts between WOs
- attendance affects financial utilization
- audit asks for project-wise historical reconciliation

Use:

- master tables
- deployment history
- attendance period/certificate
- ledger entries

That gives traceability.

24) My recommended final blueprint

Business model

This system should be treated as a:

Project-based Vendor Resource and Advance Management System

Core principles

- PO and WO are contract references
- employees are deployed through WO
- attendance drives monthly utilization
- utilization reduces advance
- ledger is source of truth for balances
- project is reporting anchor
- PO/WO are audit anchors

25) Minimum ER relationship view

